

Information-based Learning

Sections 4.4, 4.5

Dr. Mohammed Brahimi and Dr. Aicha Boutorh



- 1 **Alternative Feature Selection Metrics**
- 2 **Handling Continuous Descriptive Features**
- 3 **Predicting Continuous Targets**
- 4 **Noisy Data, Overfitting and Tree Pruning**
- 5 **Bias-Variance Tradeoff**
- 6 **Model Ensembles**
 - Voting
 - Stacking
 - Bagging
 - Boosting
 - Gradient Boosting
- 7 **Summary**

Alternative Feature Selection Metrics

- Entropy-based information gain, preferences features with many values.
- One way of addressing this issue is to use **information gain ratio** which is computed by dividing the information gain of a feature by entropy of the feature:

$$GR(d, \mathcal{D}) = \frac{IG(d, \mathcal{D})}{-\sum_{l \in \text{levels}(d)} (P(d = l) \times \log_2(P(d = l)))} \quad (1)$$

$$IG(\text{STREAM}, \mathcal{D}) = 0.3060$$

$$IG(\text{SLOPE}, \mathcal{D}) = 0.5774$$

$$IG(\text{ELEVATION}, \mathcal{D}) = 0.8774$$

$H(\text{STREAM}, \mathcal{D})$

$$\begin{aligned}
 &= - \sum_{l \in \left\{ \begin{smallmatrix} \text{true,} \\ \text{false} \end{smallmatrix} \right\}} P(\text{STREAM} = l) \times \log_2 (P(\text{STREAM} = l)) \\
 &= - \left(\left(\frac{4}{7} \times \log_2 \left(\frac{4}{7} \right) \right) + \left(\frac{3}{7} \times \log_2 \left(\frac{3}{7} \right) \right) \right) \\
 &= 0.9852 \text{ bits}
 \end{aligned}$$

 $H(\text{SLOPE}, \mathcal{D})$

$$\begin{aligned}
 &= - \sum_{l \in \left\{ \begin{smallmatrix} \text{flat,} \\ \text{moderate,} \\ \text{steep} \end{smallmatrix} \right\}} P(\text{SLOPE} = l) \times \log_2 (P(\text{SLOPE} = l)) \\
 &= - \left(\left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) + \left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) + \left(\frac{5}{7} \times \log_2 \left(\frac{5}{7} \right) \right) \right) \\
 &= 1.1488 \text{ bits}
 \end{aligned}$$

 $H(\text{ELEVATION}, \mathcal{D})$

$$\begin{aligned}
 &= - \sum_{l \in \left\{ \begin{smallmatrix} \text{low,} \\ \text{medium,} \\ \text{high,} \\ \text{highest} \end{smallmatrix} \right\}} P(\text{ELEVATION} = l) \times \log_2 (P(\text{ELEVATION} = l)) \\
 &= - \left(\left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) + \left(\frac{2}{7} \times \log_2 \left(\frac{2}{7} \right) \right) + \left(\frac{3}{7} \times \log_2 \left(\frac{3}{7} \right) \right) + \left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) \right) \\
 &= 1.8424 \text{ bits}
 \end{aligned}$$

$$GR(\text{STREAM}, \mathcal{D}) = \frac{0.3060}{0.9852} = 0.3106$$

$$GR(\text{SLOPE}, \mathcal{D}) = \frac{0.5774}{1.1488} = 0.5026$$

$$GR(\text{ELEVATION}, \mathcal{D}) = \frac{0.8774}{1.8424} = 0.4762$$

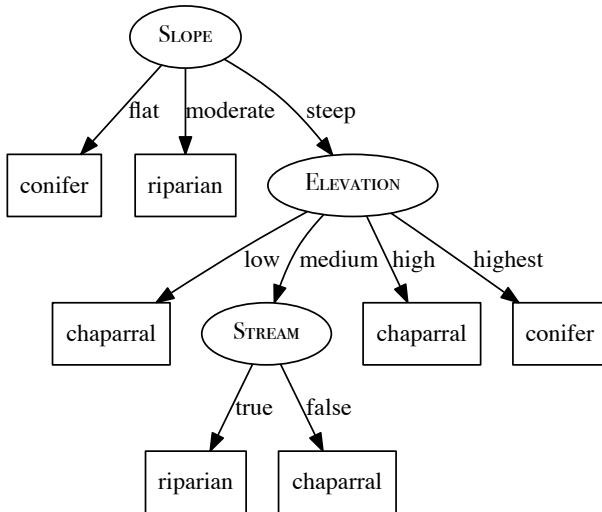


Figure 1: The vegetation classification decision tree generated using information gain ratio.

- Another commonly used measure of impurity is the **Gini index**:

$$Gini(t, \mathcal{D}) = 1 - \sum_{l \in \text{levels}(t)} P(t = l)^2 \quad (2)$$

- The Gini index can be thought of as calculating how often you would misclassify an instance in the dataset if you classified it based on the distribution of classifications in the dataset.
- The Gini gain can be calculated using the Gini index by replacing the entropy measure with the Gini index.

$$Gini(\text{VEGETATION}, \mathcal{D})$$

$$= 1 - \sum_{l \in \left\{ \begin{array}{l} \text{chapparal,} \\ \text{riparian,} \\ \text{conifer} \end{array} \right\}} P(\text{VEGETATION} = l)^2$$

$$= 1 - \left((3/7)^2 + (2/7)^2 + (2/7)^2 \right)$$

$$= 0.6531$$

Table 1: Partition sets (Part.), entropy, Gini index, remainder (Rem.), and information gain (Info. Gain) by feature

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	<i>true</i>	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	<i>false</i>	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	<i>flat</i>	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	<i>moderate</i>	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	<i>steep</i>	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
	<i>low</i>	$\mathcal{D}_6$	$\mathbf{d}_2$	0		
ELEVATION	<i>medium</i>	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5	0.3333	0.3198
	<i>high</i>	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	<i>highest</i>	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

Handling Continuous Descriptive Features

- The easiest way to handle continuous valued descriptive features is to turn them into boolean features by defining a threshold and using this threshold to partition the instances based their value of the continuous descriptive feature.
- How do we set the threshold?

- 1 The instances in the dataset are sorted according to the continuous feature values.
- 2 The adjacent instances in the ordering that have different classifications are then selected as possible threshold points.
- 3 The optimal threshold is found by computing the information gain for each of these classification transition boundaries and selecting the boundary with the highest information gain as the threshold.

- Once a threshold has been set the dynamically created new boolean feature can compete with the other categorical features for selection as the splitting feature at that node.
- This process can be repeated at each node as the tree grows.

Table 2: Dataset for predicting the vegetation in an area with a continuous ELEVATION feature (measured in feet).

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	3 900	chapparal
2	true	moderate	300	riparian
3	true	steep	1 500	riparian
4	false	steep	1 200	chapparal
5	false	flat	4 450	conifer
6	true	steep	5 000	conifer
7	true	steep	3 000	chapparal

Table 3: Dataset for predicting the vegetation in an area sorted by the continuous ELEVATION feature.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
2	true	moderate	300	riparian
4	false	steep	1 200	chapparal
3	true	steep	1 500	riparian
7	true	steep	3 000	chapparal
1	false	steep	3 900	chapparal
5	false	flat	4 450	conifer
6	true	steep	5 000	conifer

Table 4: Partition sets (Part.), entropy, remainder (Rem.), and information gain (Info. Gain) for the candidate ELEVATION thresholds: ≥ 750 , $\geq 1\,350$, $\geq 2\,250$ and $\geq 4\,175$.

Split by Threshold	Part.	Instances	Partition Entropy	Rem.	Info. Gain
≥ 750	$\mathcal{D}_1$	$\mathbf{d}_2$	0.0	1.2507	0.3060
	$\mathcal{D}_2$	$\mathbf{d}_4, \mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.4591		
$\geq 1\,350$	$\mathcal{D}_3$	$\mathbf{d}_2, \mathbf{d}_4$	1.0	1.3728	0.1839
	$\mathcal{D}_4$	$\mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.5219		
$\geq 2\,250$	$\mathcal{D}_5$	$\mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_3$	0.9183	0.9650	0.5917
	$\mathcal{D}_6$	$\mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.0		
$\geq 4\,175$	$\mathcal{D}_7$	$\mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1$	0.9710	0.6935	0.8631
	$\mathcal{D}_8$	$\mathbf{d}_5, \mathbf{d}_6$	0.0		

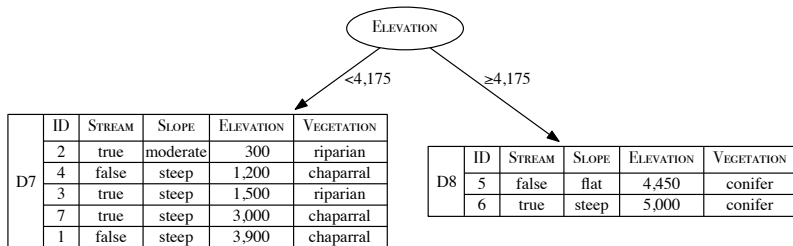


Figure 2: The vegetation classification decision tree after the dataset has been split using $ELEVATION \geq 4\,175$.

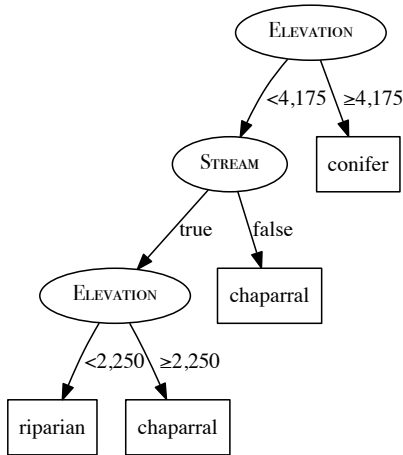


Figure 3: The decision tree that would be generated for the vegetation classification dataset listed in Table 3^[17] using information gain.

Predicting Continuous Targets

- Regression trees are constructed so as to reduce the **variance** in the set of training examples at each of the leaf nodes in the tree
- We can do this by adapting the ID3 algorithm to use a measure of variance rather than a measure of classification impurity (entropy) when selecting the best attribute

- The impurity (variance) at a node can be calculated using the following equation:

$$\text{var}(t, \mathcal{D}) = \frac{\sum_{i=1}^n (t_i - \bar{t})^2}{n - 1} \quad (3)$$

- We select the feature to split on at a node by selecting the feature that minimizes the weighted variance across the resulting partitions:

$$\mathbf{d}[\text{best}] = \arg \min_{\mathbf{d} \in \mathbf{d}} \sum_{l \in \text{levels}(\mathbf{d})} \frac{|\mathcal{D}_{d=l}|}{|\mathcal{D}|} \times \text{var}(t, \mathcal{D}_{d=l}) \quad (4)$$

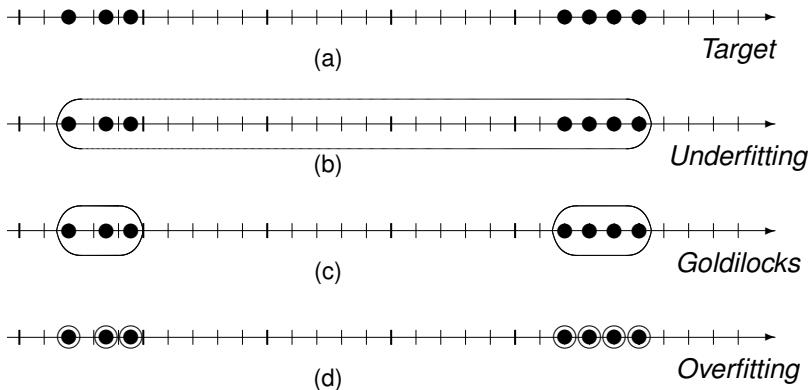


Figure 4: (a) A set of instances on a continuous number line; (b), (c), and (d) depict some of the potential groupings that could be applied to these instances.

Table 5: A dataset listing the number of bike rentals per day.

ID	SEASON	WORK DAY	RENTALS	ID	SEASON	WORK DAY	RENTALS
1	winter	false	800	7	summer	false	3000
2	winter	false	826	8	summer	true	5800
3	winter	true	900	9	summer	true	6200
4	spring	false	2100	10	autumn	false	2910
5	spring	true	4740	11	autumn	false	2880
6	spring	true	4900	12	autumn	true	2820

Table 6: The partitioning of the dataset in Table 5^[25] based on SEASON and WORK DAY features and the computation of the weighted variance for each partitioning.

Split by Feature	Level	Part.	Instances	$\frac{ \mathcal{D}_{d=l} }{ \mathcal{D} }$	$var(t, \mathcal{D})$	Weighted Variance
SEASON	winter	$\mathcal{D}_1$	$\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_3$	0.25	2 692	$1\,379\,331\frac{1}{3}$
	spring	$\mathcal{D}_2$	$\mathbf{d}_4, \mathbf{d}_5, \mathbf{d}_6$	0.25	$2\,472\,533\frac{1}{3}$	
	summer	$\mathcal{D}_3$	$\mathbf{d}_7, \mathbf{d}_8, \mathbf{d}_9$	0.25	3 040 000	
	autumn	$\mathcal{D}_4$	$\mathbf{d}_{10}, \mathbf{d}_{11}, \mathbf{d}_{12}$	0.25	2 100	
WORK DAY	true	$\mathcal{D}_5$	$\mathbf{d}_3, \mathbf{d}_5, \mathbf{d}_6, \mathbf{d}_8, \mathbf{d}_9, \mathbf{d}_{12}$	0.50	$4\,026\,346\frac{1}{3}$	$2\,551\,813\frac{1}{3}$
	false	$\mathcal{D}_6$	$\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_7, \mathbf{d}_{10}, \mathbf{d}_{11}$	0.50	1 077 280	

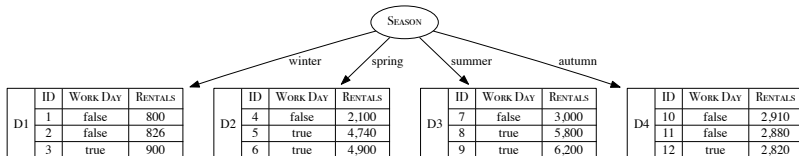


Figure 5: The decision tree resulting from splitting the data in Table 5^[25] using the feature SEASON.

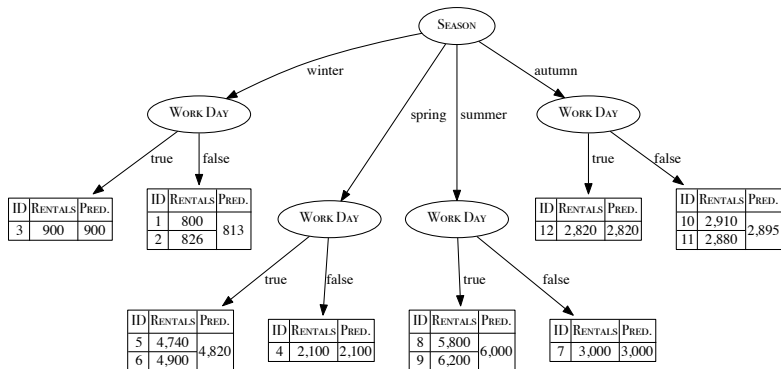


Figure 6: The final decision tree induced from the dataset in Table 5^[25]. To illustrate how the tree generates predictions, this tree lists the instances that ended up at each leaf node and the prediction (PRED.) made by each leaf node.

Noisy Data, Overfitting and Tree Pruning

- In the case of a decision tree, over-fitting involves splitting the data on an irrelevant feature.

The likelihood of over-fitting occurring increases as a tree gets deeper because the resulting classifications are based on smaller and smaller subsets as the dataset is partitioned after each feature test in the path.

- **Pre-pruning**: stop the recursive partitioning early. Pre-pruning is also known as **forward pruning**.

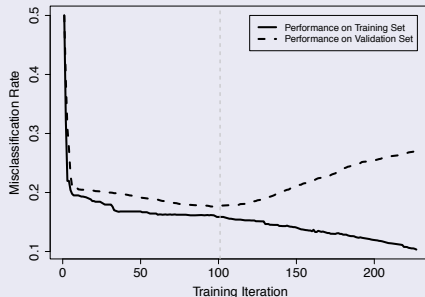
Common **Pre**-pruning Approaches

- 1 early stopping
- 2 χ^2 pruning

- **Post-pruning**: allow the algorithm to grow the tree as much as it likes and then prune the tree of the branches that cause over-fitting.

Common **Post**-pruning Approach

- Using the validation set evaluate the prediction accuracy achieved by both the fully grown tree and the pruned copy of the tree. If the pruned copy of the tree performs no worse than the fully grown tree the node is a candidate for pruning.



Chi-square Test Example: Spam Prediction

ID	Suspicious Words	Unknown Sender	Contains Images	Class
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Step 1: Constructing Observed Frequencies

- Analyze the relationship between "CONTAINS IMAGES" and "CLASS" (spam or ham).
- Observed frequency table:

CONTAINS IMAGES			
	ham (E)	spam (E)	
False	2	2	4
True	1	1	2
Total	3	3	6

- This table shows the distribution of emails with or without images across two classes.

Step 2: Calculating Expected Frequencies

- Expected frequency formula: $E_{ij} = \frac{(\text{Row Total}) \times (\text{Column Total})}{\text{Grand Total}}$
- Calculation for each cell:

CONTAINS			
IMAGES		ham (E)	spam (E)
False	$E_{False,ham} = \frac{4 \times 3}{6} = 2$		$E_{False,spam} = \frac{4 \times 3}{6} = 2$
True	$E_{True,ham} = \frac{2 \times 3}{6} = 1$		$E_{True,spam} = \frac{2 \times 3}{6} = 1$

- These values represent the expected distribution if there was no association between the variables.

Step 3: Computing χ^2 Statistic and Conclusion

- Chi-square formula: $\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$
- With observed and expected frequencies equal, $\chi^2 = 0$.
- Degrees of freedom (df) = $(2 - 1) \times (2 - 1) = 1$ (for a 2x2 table).
- P-value interpretation:
 - A p-value of 1.0 suggests no statistical evidence of the association between "CONTAINS IMAGES" and "CLASS".
- Conclusion: The test indicates there is no significant relationship, suggesting that the presence of images does not influence the classification as spam or ham.

P-Value is the probability that the pattern you see in your data happened by pure luck.
The relationship between the Chi-square value and the p-value is inverse:

Table 7: An example validation set for the post-operative patient routing task.

ID	CORE- TEMP	STABLE- TEMP	GENDER	DECISION
1	high	true	male	gen
2	low	true	female	icu
3	high	false	female	icu
4	high	false	male	icu
5	low	false	female	icu
6	low	true	male	icu

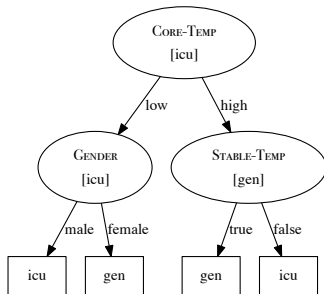


Figure 7: The decision tree for the post-operative patient routing task.

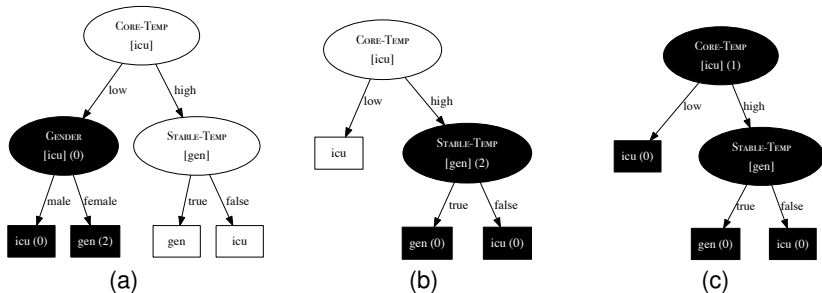


Figure 8: The iterations of reduced error pruning for the decision tree in Figure 7^[38] using the validation set in Table 7^[37]. The subtree that is being considered for pruning in each iteration is highlighted in black. The prediction returned by each non-leaf node is listed in square brackets. The error rate for each node is given in round brackets.

Advantages of pruning:

- Smaller trees are easier to interpret
- Increased generalization accuracy when there is noise in the training data (**noise dampening**).

Bias-Variance Tradeoff

Decomposition of Expected Test Error

$$\underbrace{\mathbb{E}_{\mathbf{x}, y, D} \left[(h_D(\mathbf{x}) - y)^2 \right]}_{\text{Expected Test Error}} = \underbrace{\mathbb{E}_{\mathbf{x}, D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2 \right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathbf{x}, y} \left[(\bar{y}(\mathbf{x}) - y)^2 \right]}_{\text{Noise}} + \underbrace{\mathbb{E}_{\mathbf{x}} \left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2 \right]}_{\text{Bias}^2} \quad (5)$$

- $\mathbb{E}_{\mathbf{x}, y, D}$: Expectation over dataset D , inputs $\mathbf{x}$, and outputs y .
- $h_D(\mathbf{x})$: A hypothesis or model trained on dataset D .
- $\bar{h}(\mathbf{x})$: The average prediction of $h_D(\mathbf{x})$ over different datasets D .

For more details, visit: **Cornell University CS4780**

Decomposition of Expected Test Error

Important!

$$\text{Expected Test Error} = \text{Variance} + \text{Noise} + \text{Bias}^2$$

- **Variance:** Reflects the model's sensitivity to different training datasets. It measures the classifier's fluctuation from one training set to another, indicating potential overfitting.
- **Bias:** Represents the error from erroneous assumptions in the learning algorithm. It measures the gap between the best possible prediction and the actual model's capabilities, highlighting underfitting.
- **Noise:** Denotes the inherent randomness present in the data itself. This irreducible error stems from the data distribution and cannot be eliminated by the model.

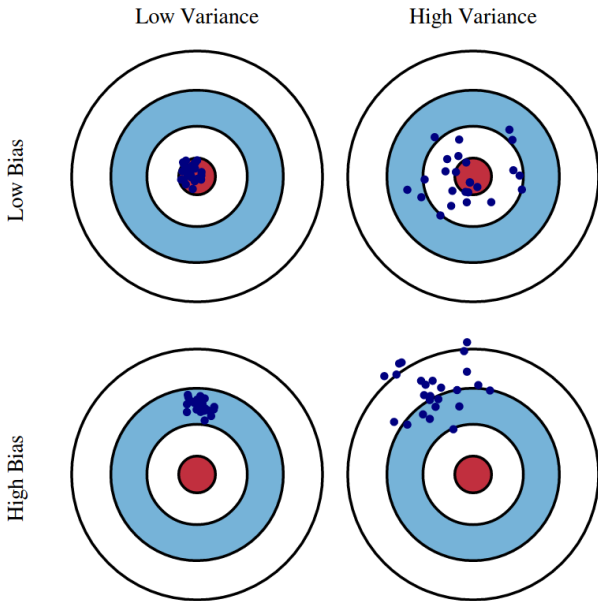


Figure 9: Graphical illustration of bias and variance.

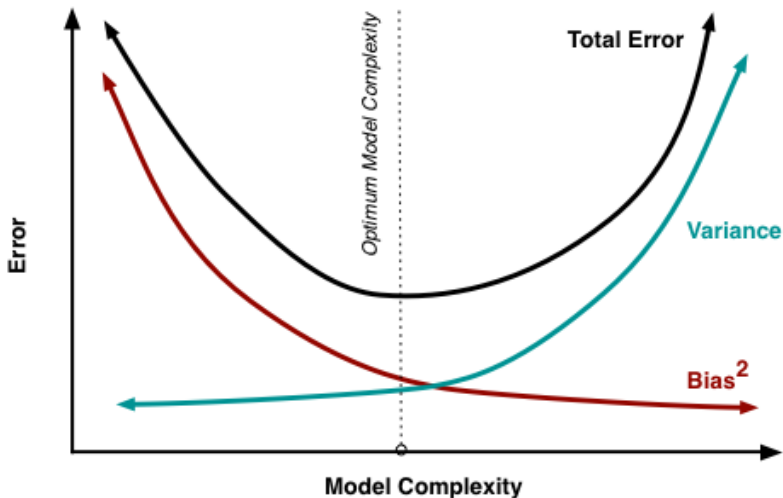


Figure 10: Variation of Bias and Variance with the model complexity. This is similar to the concept of overfitting and underfitting. More complex models overfit while the simplest models underfit.

Detecting High Bias and High Variance

High Bias (Underfitting)

- Performs poorly on training/test data.
- Fails to capture data trends.
- **Indicators:** Low training accuracy.
- **Solutions:** Increase complexity; add features.

Detecting High Bias and High Variance

High Bias (Underfitting)

- Performs poorly on training/test data.
- Fails to capture data trends.
- **Indicators:** Low training accuracy.
- **Solutions:** Increase complexity; add features.

High Variance (Overfitting)

- Good on training data, poor on test data.
- Model too complex, capturing noise.
- **Indicators:** Large gap between training and validation error.
- **Solutions:** Simplify model; add more data.

Model Ensembles

Why Model Ensembles?



- A committee of experts, by working collaboratively, enhances problem-solving success beyond individual capabilities.
- These models aim for accuracy and robustness, leveraging diverse strengths while minimizing individual errors.

Why Model Ensembles?

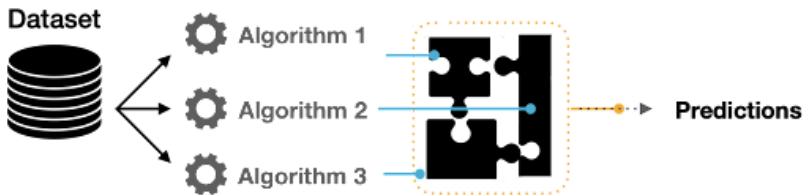


- A committee of experts, by working collaboratively, enhances problem-solving success beyond individual capabilities.
- These models aim for accuracy and robustness, leveraging diverse strengths while minimizing individual errors.
- Avoiding **groupthink** is crucial; it's a barrier to critical reasoning and exploring alternatives. weaknesses.

Why Model Ensembles?

- **Goal:** Improve accuracy and robustness by balancing error sources.
- **Bias Reduction:** Combines simpler models to address underfitting, enhancing the overall model's ability to capture data trends.
- **Variance Reduction:** Averages multiple model outputs to counteract overfitting, ensuring stability across different data samples.

What is Model Ensembles?



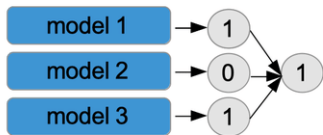
- Rather than creating a single model they generate a set of models and then make predictions by aggregating the outputs of these models.
- A prediction model that is composed of a set of models is called a **model ensemble**.
- For this approach to work the models that are in the ensemble must be different from each other.

How Model Ensembles Works?

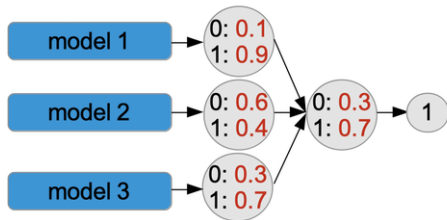
- There are standard approaches to creating ensembles:
 - 1 **Voting**
 - 2 **Stacking**
 - 3 **Bagging.**
 - 4 **Boosting**

Voting in Ensemble Learning

- Voting combines the predictions of multiple models to make a final prediction.
- Two main types:
 - Hard voting (majority vote): selects the prediction with the most votes from models.
 - Soft voting (weighted probability): averages the predicted probabilities, favoring higher-confidence predictions.
- Enhances prediction accuracy and model robustness by leveraging diverse models.



Hard voting

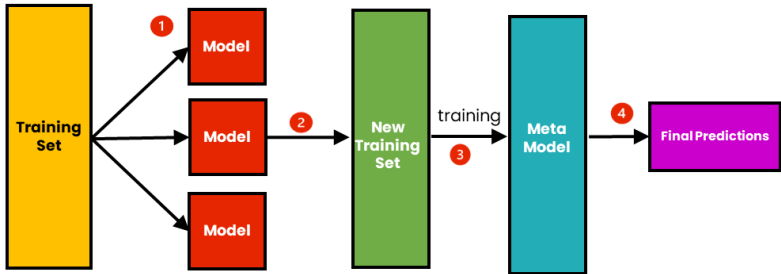


Soft voting

Stacking

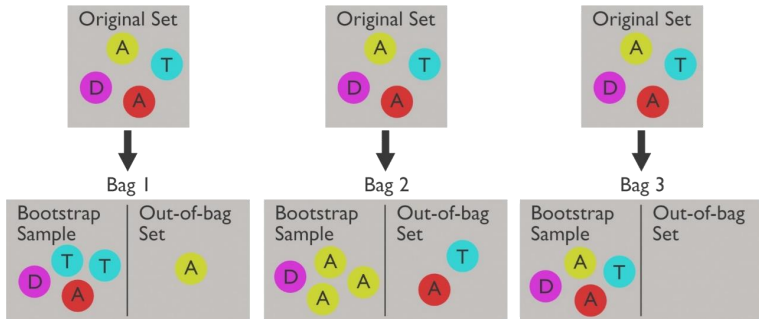
- Stacking combines multiple models to improve prediction accuracy.
- It uses a **meta-model** to learn the best way to combine predictions from several base models.
- Base models are trained on the full dataset, then the meta-model is trained on the outputs of the base models as features.
- This method leverages the strengths of various models to achieve higher performance than any single model.
- Effective for both classification and regression tasks.

The Process of Stacking



Bagging

- When we use **bagging** (or **bootstrap aggregating**) each model in the ensemble is trained on a random sample of the dataset known as **bootstrap samples**.
- Each random sample is the same size as the dataset and **sampling with replacement** is used.
- Each Bootstrap sample misses some instances, ensuring variability among samples. Consequently, trained models will differ.



Training data

Class	A	B	C
1	a1	b1	c1
2	a2	b2	c2
2	a3	b3	c3
1	a4	b4	c4
2	a5	b5	c5

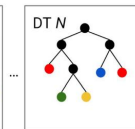
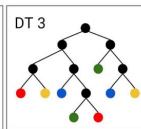
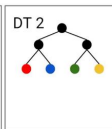
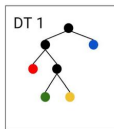
Bootstrap

Class	A	B	C
1	a1	b1	c1
2	a2	b2	c2
2	a3	b3	c3
1	a4	b4	c4
1	a4	b4	c4

Class	A	B	C
2	a2	b2	c2
2	a2	b2	c2
2	a3	b3	c3
2	a5	b5	c5
1	a1	b1	c1

Class	A	B	C
1	a4	b4	c4
2	a5	b5	c5
1	a1	b1	c1
2	a5	b5	c5
2	a3	b3	c3

Ensemble of trees



Class 1

Class 2

Class 1

Class 1

Majority vote: Class 1

Subspace Sampling

- **subspace sampling**: bagging and each bootstrap sample only uses a randomly selected subset of the descriptive features.
- The combination of bagging, subspace sampling, and decision trees is known as a **random forest** model.

Bagging

Random Forest (RF)

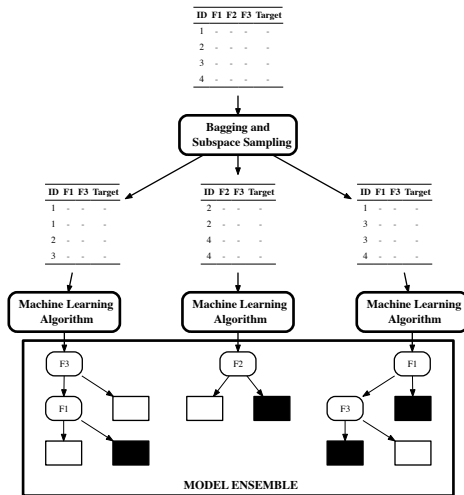
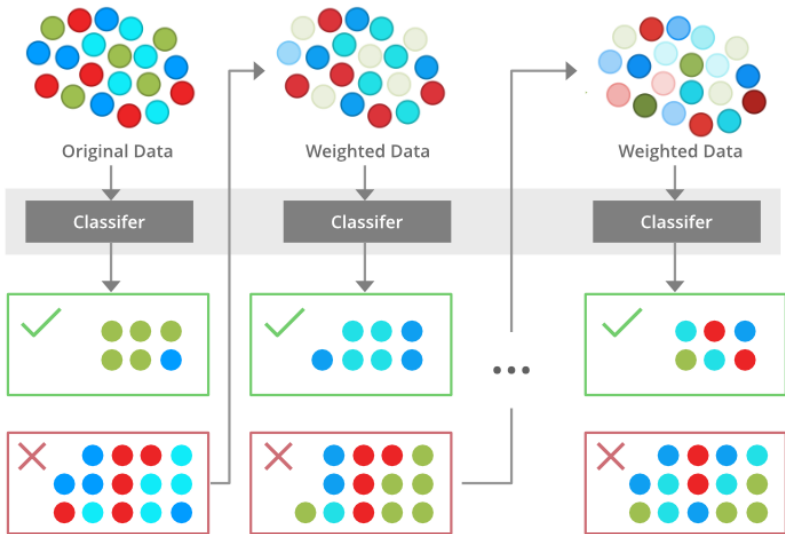


Figure 11: The process of creating a model ensemble using bagging

Boosting

- Boosting iteratively creates models and adds them to the ensemble.
- The iteration stops when a predefined number of models have been added.
- When we use **boosting** each newly added model pays more attention to miss-classified instances in the previous models.
- This is done by incrementally adapting the dataset used to train the models. To do this we use a **weighted dataset**



Weighted Dataset

- Each instance has an associated weight $w_i \geq 0$,
- Initially set to $\frac{1}{n}$ where n is the number of instances in the dataset.
- After each model is added to the ensemble it is tested on the **training data**:
 - The weights of the instances the model gets correct are decreased.
 - The weights of the instances the model gets incorrect are increased.
- The weights are used as a distribution over which the dataset is sampled to create a **replicated training set**, where the replication of an instance is proportional to its weight.

AdaBoost

- AdaBoost (Adaptive Boosting) combining multiple **weak learners** into a **strong one**.
- Iteratively adjusts the weights of incorrectly classified instances, making them more likely to be correctly classified in the next round.
- Focuses on difficult cases, improving the model's performance on varied datasets.
- Utilized in both binary and multi-class classification problems.
- Adaboost is one of the **top 10 ML algorithms**.

Adaboost

During each **training iteration** the algorithm:

- 1 Induces a model and calculates the total error, ϵ , by summing the weights of the training instances for which the predictions made by the model are incorrect.
- 2 Increases the weights for the instances misclassified using:

$$\mathbf{w}[i] \leftarrow \mathbf{w}[i] \times \left(\frac{1}{2 \times \epsilon} \right) \quad (6)$$

- 3 Decreases the weights for the instances correctly classified:

$$\mathbf{w}[i] \leftarrow \mathbf{w}[i] \times \left(\frac{1}{2 \times (1 - \epsilon)} \right) \quad (7)$$

- 4 Calculate a **confidence factor**, α , for the model such that α increases as ϵ decreases:

$$\alpha = \frac{1}{2} \times \log_e \left(\frac{1 - \epsilon}{\epsilon} \right) \quad (8)$$

AdaBoost: Model Aggregation

Predictions

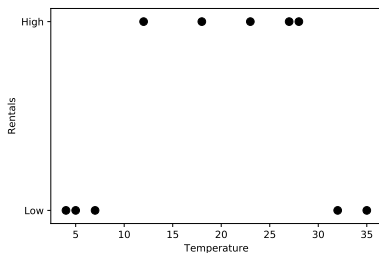
- The ensemble predicts by combining individual models' outputs through a weighted vote.

Weights

- Aggregation weights are based on each model's prediction confidence.

Table 8: A simple bicycle demand predictions dataset and the workings of the first three iterations of training an ensemble model using boosting to predict RENTALS given TEMP.

ID	TEMP	RENTALS	Iteration 0			Iteration 1			Iteration 2		
			Dist.	Freq.	$M_0(\mathbf{d})$	Dist.	Freq.	$M_1(\mathbf{d})$	Dist.	Freq.	$M_2(\mathbf{d})$
1	4	Low	0.100	2	Low	0.062	0	High	0.167	2	Low
2	5	Low	0.100	1	Low	0.062	1	High	0.167	1	Low
3	7	Low	0.100	0	Low	0.062	1	High	0.167	3	Low
4	12	High	0.100	1	High	0.062	2	High	0.038	0	Low
5	18	High	0.100	1	High	0.062	0	High	0.038	0	Low
6	23	High	0.100	1	High	0.062	0	High	0.038	0	Low
7	27	High	0.100	1	High	0.062	1	High	0.038	0	Low
8	28	High	0.100	1	High	0.062	1	High	0.038	1	Low
9	32	Low	0.100	2	High	0.250	3	Low	0.154	1	Low
10	35	Low	0.100	0	High	0.250	1	Low	0.154	2	Low



(a) Training data

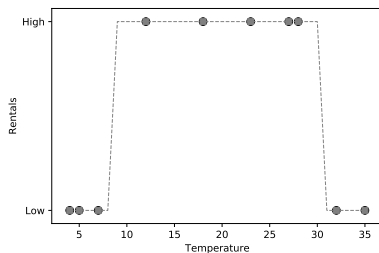
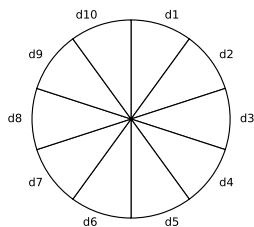
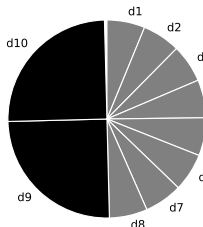
(b) The final ensemble model, M

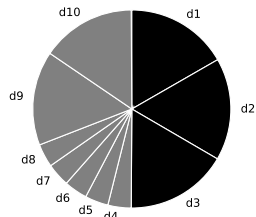
Figure 12: (a) A plot of the bike rental dataset from Table 8^[69]. (b) An illustration of the final ensemble model trained using the boosting algorithm. (c)–(e) A representation of the changing weights used to generate sample datasets for the first iterations of the boosting process.



(c) Distribution 0



(d) Distribution 1



(e) Distribution 2

Figure 13: (a) A plot of the bike rental dataset from Table 8^[69]. (b) An illustration of the final ensemble model trained using the boosting algorithm. (c)–(e) A representation of the changing weights used to generate sample datasets for the first iterations of the boosting process.

$$\mathbf{w}[1] \leftarrow 0.100 \times \left(\frac{1}{2 \times (1 - 0.200)} \right) \leftarrow 0.0625$$

$$\mathbf{w}[9] \leftarrow 0.100 \times \left(\frac{1}{2 \times 0.200} \right) \leftarrow 0.250$$

Gradient Boosting

- Gradient Boosting combines simple models into a complex one by additive combination.
- Individual models may poorly fit the data, yet their linear combination enhances expressiveness and flexibility.

- $$M = \sum_{h \in H} \lambda_h M_h$$

Key Question

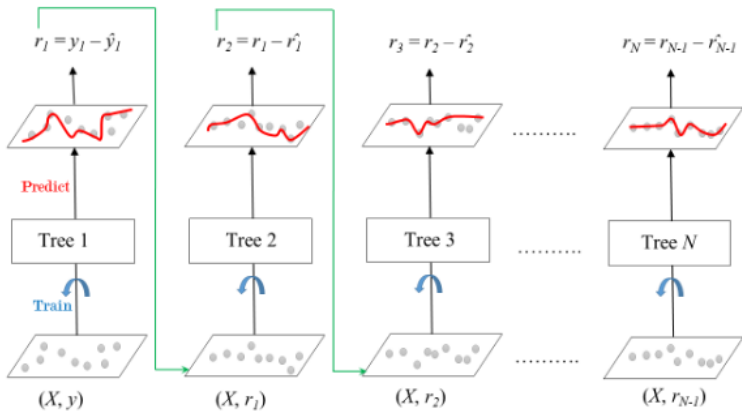
How to select models for the ensemble and determine their combination weights?

Gradient Boosting

Gradient boosting adds iteratively a simple model to the ensemble to **compensates for the weaknesses of the current ensemble**.

- 1 Fit a simple model $M(x, y)$ on the training data $\{(x_0, y_0), \dots, (x_N, y_N)\}$. Set $M \leftarrow M(x, y)$. Compute residuals $\{r_1, \dots, r_N\}$ for M .
- 2 Fit a simple model, $M^{(1)}$, to the current residuals, i.e., train using $\{(x_0, r_0), \dots, (x_N, r_N)\}$.
- 3 Update $M \leftarrow M + \alpha M^{(1)}$.
- 4 Compute new residuals, set $r_i \leftarrow r_i - \alpha M^{(1)}(x_i)$; for $i = 1, \dots, N$.
- 5 Repeat steps 2-4 until the stopping condition is met.

Where α is a constant called the **learning rate**.



$$\mathbb{M}_0(\mathbf{d}) = \frac{1}{n} \sum_i t_i \quad (9)$$

$$\mathbb{M}_1(\mathbf{d}) = \mathbb{M}_0(\mathbf{d}) + \mathbb{M}_{\Delta 1}(\mathbf{d}) \quad (10)$$

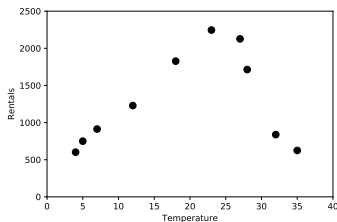
$$\mathbb{M}_i(\mathbf{d}) = \mathbb{M}_{i-1}(\mathbf{d}) + \mathbb{M}_{\Delta i}(\mathbf{d}) \quad (11)$$

$$\begin{aligned}
 \mathbb{M}_4(\mathbf{d}) &= \mathbb{M}_3(\mathbf{d}) + \mathbb{M}_{\Delta 4}(\mathbf{d}) \\
 &= (\mathbb{M}_2(\mathbf{d}) + \mathbb{M}_{\Delta 3}(\mathbf{d})) + \mathbb{M}_{\Delta 4}(\mathbf{d}) \\
 &= ((\mathbb{M}_1 + \mathbb{M}_{\Delta 2}(\mathbf{d})) + \mathbb{M}_{\Delta 3}(\mathbf{d})) + \mathbb{M}_{\Delta 4}(\mathbf{d}) \\
 &= (((\mathbb{M}_0(\mathbf{d}) + \mathbb{M}_{\Delta 1}(\mathbf{d})) + \mathbb{M}_{\Delta 2}(\mathbf{d})) + \mathbb{M}_{\Delta 3}(\mathbf{d})) + \mathbb{M}_{\Delta 4}(\mathbf{d}) \\
 &= \mathbb{M}_0(\mathbf{d}) + \mathbb{M}_{\Delta 1}(\mathbf{d}) + \mathbb{M}_{\Delta 2}(\mathbf{d}) + \mathbb{M}_{\Delta 3}(\mathbf{d}) + \mathbb{M}_{\Delta 4}(\mathbf{d})
 \end{aligned}
 \tag{12}$$

Table 9: A simple bicycle demand predictions dataset and the workings of the first iterations of training a gradient boosting model.

ID	TEMP	RENTALS	$M_0(\mathbf{d})$	$t - M_0(\mathbf{d})$	$M_{\Delta 1}(\mathbf{d})$	$M_1(\mathbf{d})$	$t - M_1(\mathbf{d})$	$M_{\Delta 2}(\mathbf{d})$	$M_2(\mathbf{d})$	$t - M_2(\mathbf{d})$	$M_{\Delta 3}(\mathbf{d})$	$M_3(\mathbf{d})$
1	4	602	1287.1	-685.1	-460.9	826.2	-224.2	-167.2	659.0	-57.0	-34.1	624.9
2	5	750	1287.1	-537.1	-460.9	826.2	-76.2	-167.2	659.0	91.0	-34.1	624.9
3	7	913	1287.1	-374.1	-460.9	826.2	86.8	71.6	897.8	15.2	-34.1	863.7
4	12	1229	1287.1	-58.1	-460.9	826.2	402.8	71.6	897.8	331.2	-34.1	863.7
5	18	1827	1287.1	539.9	691.4	1978.5	-151.5	71.6	2050.1	-223.1	-34.1	2016.1
6	23	2246	1287.1	958.9	691.4	1978.5	267.5	71.6	2050.1	195.9	136.4	2186.5
7	27	2127	1287.1	839.9	691.4	1978.5	148.5	71.6	2050.1	76.9	136.4	2186.5
8	28	1714	1287.1	426.9	691.4	1978.5	-264.5	71.6	2050.1	-336.1	-34.1	2016.1
9	32	838	1287.1	-449.1	-460.9	826.2	11.8	71.6	897.8	-59.8	-34.1	863.7
10	35	625	1287.1	-662.1	-460.9	826.2	-201.2	-167.2	659.0	-34.0	-34.1	624.9

Gradient Boosting



(a) Training data

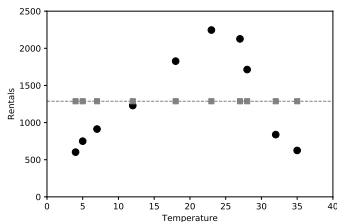
(b) M_0

Figure 14: (a) A plot of the bike rental dataset from Table 9^[78]. (b)–(e) Visualizations of the prediction models trained in the early iterations of the gradient boosting process. (f) The final ensemble model trained after 20 iterations of gradient boosting.

Gradient Boosting

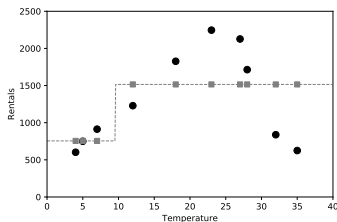
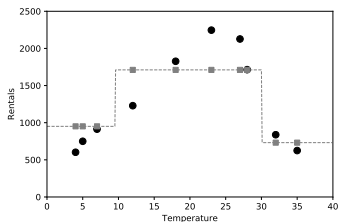
(c) M_1 (d) M_2

Figure 15: (a) A plot of the bike rental dataset from Table 9^[78]. (b)–(e) Visualizations of the prediction models trained in the early iterations of the gradient boosting process. (f) The final ensemble model trained after 20 iterations of gradient boosting.

Gradient Boosting

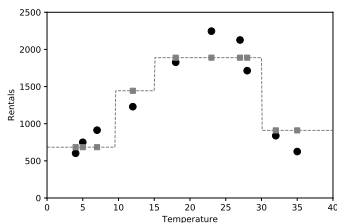
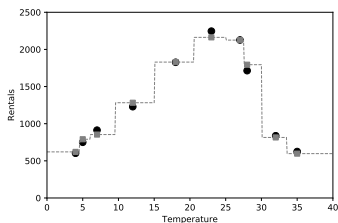
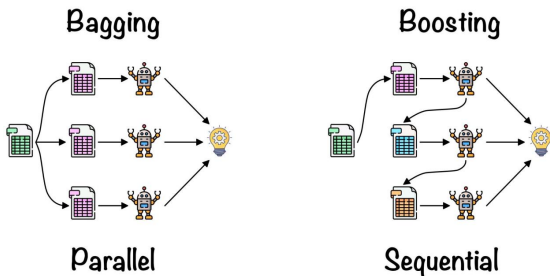
(e) M_3 (f) M_{20}

Figure 16: (a) A plot of the bike rental dataset from Table 9^[78]. (b)–(e) Visualizations of the prediction models trained in the early iterations of the gradient boosting process. (f) The final ensemble model trained after 20 iterations of gradient boosting.

Bagging Vs Boosting



- Bagging is simpler and easier to parallelize.
- Empirical results indicate:
 - Boosted decision trees perform best on datasets containing up to 4,000 features.
 - Random forest ensembles (based on bagging) performed better for datasets containing more than 4,000 features.

Summary

Decision Trees: Advantages

- interpretable.
- handles both categorical and continuous descriptive features.
- can model the interactions between descriptive features (diminished if **pre-pruning** is employed)
- relatively, robust to the **curse of dimensionality**.
- relatively, robust to noise in the dataset if **pruning** is used.

Decision Tress: Potential Disadvantages

- trees become large when dealing with continuous features.
- decision trees are very expressive and sensitive to the dataset, as a result, they can overfit the data if there are a lot of features (curse of dimensionality)
- eager learner (concept drift).

- 1 **Alternative Feature Selection Metrics**
- 2 **Handling Continuous Descriptive Features**
- 3 **Predicting Continuous Targets**
- 4 **Noisy Data, Overfitting and Tree Pruning**
- 5 **Bias-Variance Tradeoff**
- 6 **Model Ensembles**
 - Voting
 - Stacking
 - Bagging
 - Boosting
 - Gradient Boosting
- 7 **Summary**

Slide Acknowledgment

The slides used in this course are based on the official textbook materials, with modifications made where necessary to suit the course requirements and enhance the learning experience.